Εισάγει τη συσχέτιση μεταξύ key και value. Αν η συσχέτιση υπάρχει ήδη, τότε την ενημερώνει και επιστρέφει την τιμή V που βρέθηκε στο MyHashMap. Αν δεν υπάρχει, την εισάγει και επιστρέφει null.

```
public V get(K key)
```

Επιστρέφει την τιμή V που σχετίζεται με αυτό το κλειδί. Αν το κλειδί δεν βρεθεί, επιστρέφει null.

```
public V remove(K key)
```

Διαγράφει από αυτό το MyHashMap τη συσχέτιση με κλειδί key. Αν η συσχέτιση δεν βρεθεί, επιστρέφει null αλλιώς τη συσχέτιση που διεγράφη.

```
public boolean isEmpty()
```

Επιστρέφει true αν καμία συσχέτιση δεν υπάρχει σε αυτό το MyHshMap.

Προσθέστε main στην οποία δημιουργήστε ένα αντικείμενο MyHashMap<String, String>. Εισάγεται μερικές συσχετίσεις. Ρυθμίστε την αρχική χωρητικότητα του εσωτερικού πίνακα και τον συντελεστή φόρτωσης έτσι ώστε με την εισαγωγή των συσχετίσεων να προκληθεί επανακατακερματισμός. Διαβάστε τις συσχετίσεις με την get. Στη συνέχεια, διαγράψτε τις συσχετίσεις μια προς μία με την remove. Μετά από κάθε διαγραφή, καλέστε την isEmpty. Ελέγξτε ώστε όλα τα αποτελέσματα να είναι τα αναμενόμενα.

Λύση

```
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public class MyHashMap<K, V> {

    public class Entry<K, V> {

        private final K key;
        private V value;

        public Entry(K key, V value) {
            this.key = key;
            this.value = value;
        }

        public K getKey() {
            return key;
        }

        public V getValue() {
            return value;
        }

        public void setValue(V value) {
            this.value = value;
        }

        @Override
        public int hashCode() {
            int hash = 7;
            hash = 97 * hash + Objects.hashCode(this.key);
            return hash;
        }

        @Override
        public boolean equals(Object obj) {
            if (this == obj) {
                return true;
```

```
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final Entry<K, V> other = (Entry<K, V>) obj;
        return Objects.equals(this.key, other.key);
    }
}

private int find(K key, List<Entry<K, V>> list) {
    for (int i = 0; i < list.size(); i++) {

        Entry<K, V> entry = list.get(i);
        if (entry.key.equals(key)) {
            return i;
        }
    }
    return -1;
}

private int size = 0;
private static final double LOADFACTOR = 0.75;
private int noOfLists = 0;
private final int INITIALCAPACITY = 3;
private int capacity = INITIALCAPACITY;
private ArrayList<Entry<K,V>>[] store = new ArrayList[INITIALCAPACITY];

public void clear() {
    store = new ArrayList[INITIALCAPACITY];
    size = 0;
    capacity = INITIALCAPACITY;
    noOfLists = 0;
}

private void rehash() {
    int newCapacity = 2 * capacity;
    int newEntries = 0;
    ArrayList<Entry<K,V>>[] local = new ArrayList[newCapacity];
    for (ArrayList<Entry<K,V>> list : store) {

        if (list != null) {
            for (Entry<K,V> entry : list) {
                                int hashCode = entry.key.hashCode();
                int idx = hashCode % newCapacity;
                if (local[idx] == null) {
                    local[idx] = new ArrayList<Entry<K, V>>();
                    newEntries++;
                }
                local[idx].add(entry);
            }
        }
    }
    store = local;
    noOfLists = newEntries;
    capacity = newCapacity;
}

public V put(K key, V value) {
```

```
        if (noOfLists > LOADFACTOR * capacity) {
            rehash();
        }
        int idx = key.hashCode() % store.length;
        if (store[idx] == null) {
            store[idx] = new ArrayList<Entry<K, V>>();
            noOfLists++;
        }
        Entry<K, V> entry = new Entry<>(key, value);
        int location = store[idx].indexOf(entry);
        Entry<K, V> oldEntry;
        if (location < 0) {
            store[idx].add(entry);
            size++;
            return null;
        } else {
            oldEntry = store[idx].get(location);
            store[idx].set(location, entry);
        }
        return oldEntry.getValue();
    }

    public V get(K key) {
        int idx = key.hashCode() % store.length;

        int location = find(key, store[idx]);
        if (location < 0) {
            return null;
        }
        return store[idx].get(location).getValue();
    }

    public V remove(K key) {
        int idx = key.hashCode() % store.length;
        int location = find(key, store[idx]);
        if (location < 0) {
            return null;
        }
        size--;
        V rVal = store[idx].remove(location).getValue();
        if (store[idx].isEmpty()) {
            noOfLists--;
            store[idx] = null;
        }
        return rVal;
    }

    public boolean isEmpty() {
        return size==0;
    }

    public static void main(String[] args) {
        MyHashMap<String, String> hM = new MyHashMap<>();
        hM.put("key1", "value1");
        hM.put("key2", "value2");
        hM.put("key3", "value3");
        hM.put("key4", "value4");
        hM.put("key5", "value5");
        hM.put("key6", "value6");
        hM.put("key7", "value7");
        hM.put("key8", "value8");
```

```
            System.out.println(hM.get("key1"));
            System.out.println(hM.get("key2"));
            System.out.println(hM.get("key3"));
            System.out.println(hM.get("key4"));
            System.out.println(hM.get("key5"));
            System.out.println(hM.get("key6"));
            System.out.println(hM.get("key7"));
            System.out.println(hM.get("key8"));

            System.out.println(hM.remove("key1"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key2"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key3"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key4"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key5"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key6"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key7"));
            System.out.println(hM.isEmpty());
            System.out.println(hM.remove("key8"));
            System.out.println(hM.isEmpty());
        }
    }
```

Κώδικας 17.13 *Η κλάση MyHashMap<K,V>*

Η κλάση MyHashMap<K,V> στον κώδικα 17.11 ορίζει την εσωτερική κλάση Entry<K,V>. Τα αντικείμενα Entry<K,V> συνιστούν τις συσχετίσεις που εισάγονται στο MyHashMap. Από τη στιγμή που δημιουργείται μία τέτοια συσχέτιση, το κλειδί της δεν μπορεί να μεταβληθεί. Για αυτόν τον λόγο το μέλος key δηλώθηκε ως final και δεν παρέχεται αντίστοιχος ρυθμιστής. Καθώς στο MyHashMap δεν επιτρέπεται η εισαγωγή συσχετίσεων με το ίδιο κλειδί, η equals θεωρεί συσχετίσεις με το ίδιο κλειδί ως ίσες, οπότε και η hashCode βασίζεται αποκλειστικά στο κλειδί.

Η κλάση διαθέτει τα μέλη δεδομένα, size που διατηρεί τον αριθμό των καταχωρημένων συσχετίσεων, LOADFACTOR που εκφράζει τον συντελεστή φόρτωσης, noOfLists που διατηρεί τον αριθμό των ενεργών λιστών, INITIALCAPACITY που ρυθμίζει την αρχική χωρητικότητα του εσωτερικού πίνακα, capacity που διατηρεί την τρέχουσα χωρητικότητα του εσωτερικού πίνακα και τον εσωτερικό πίνακα store. Προσέξτε πως ο store ως προς τον τύπο είναι πίνακας από ArrayList<Entry<K,V>>. Ωστόσο ο πίνακας που δημιουργείται είναι πρωταρχικός τύπος, καθώς η δημιουργία πινάκων γενικεύσεων δεν υποστηρίζεται. Επομένως, η δημιουργία του store είναι αναγκαστικά μη ασφαλής λειτουργία. Ωστόσο, η κλάση οργανώνεται κατά τρόπο που δεν υποχρεώνει τον χρήστη της σε μη ασφαλείς λειτουργίες. Αντίθετα, όλες οι μη ασφαλείς λειτουργίες είναι εσωτερικές στην κλάση.

Η find είναι μια ιδιωτική συνάρτηση που ψάχνει σε μία λίστα να βρει τη συσχέτιση με κλειδί key. Αν την βρει επιστρέφει τη θέση της στη λίστα, αν όχι επιστρέφει -1.

Η clear δημιουργεί νέο εσωτερικό χώρο και αρχικοποιεί κατάλληλα τις μεταβλητές size, capacity, και noOfLists.

Η put τοποθετεί μία συσχέτιση σε αυτό το MyHashMap. Η πρώτη ενέργεια της put είναι να ελέγξει τον αριθμό των ενεργών λιστών, δηλαδή τον αριθμό των κατειλημμένων θέσεων του πίνακα store. Αν αυτός υπερβαίνει το γινόμενο LOADFACTOR*capacity, τότε καλεί την rehash για να μεγεθύνει τον εσωτερικό αποθηκευτικό χώρο και να επανατοποθετήσει τις συσχετίσεις. Στη συνέχεια εντοπίζει το κλειδί κατακερματισμού της key και υπολογίζει τη θέση του store στην οποία πρέπει να αποθηκευτεί η συσχέτιση. Αν σε εκείνη τη θέση του πίνακα δεν υπάρχει λίστα, τη δημιουργεί και αυξάνει κατάλληλα την noOfLists. Μετά δημιουργεί τη συσχέτιση ως Entry, ελέγχει αν υπάρχει ήδη στην κατάλληλη λίστα και αν υπάρχει την ενημερώνει και επιστρέφει την παλιά τιμή, αλλιώς την εισάγει και επιστρέφει null. Στην περίπτωση που την εισάγει ενημερώνει και τη μεταβλητή size.

Η get επιστρέφει την τιμή που συσχετίζεται με το κλειδί που λαμβάνει ως παράμετρο. Αν δεν υπάρχει συσχέτιση για αυτό το κλειδί, επιστρέφει null.

Η remove διαγράφει τη συσχέτιση στην οποία αναφέρεται το κλειδί που λαμβάνει ως παράμετρο και μειώνει το size κατά 1, εφόσον βρεθεί το κλειδί. Αν με τη διαγραφή της συσχέτισης η αντίστοιχη λίστα αδειάσει, τότε θέτει τη θέση της στον πίνακα στο null και μειώνει την noOfLists κατά 1.

Η isEmpty βασίζεται στη μεταβλητή empty που ενημερώνεται με κάθε εισαγωγή και διαγραφή συσχετίσεων από αυτό το MyHashMap.

Τέλος, η rehash δημιουργεί έναν νέο πίνακα, επανατοποθετεί τα στοιχεία του store στο νέο πίνακα και ανακατευθύνει τη μεταβλητή store ώστε να αναφέρεται στον νέο πίνακα. Ο παλαιότερος πίνακας μένει χωρίς αναφορά και είναι διαθέσιμος για διαγραφή από τον συλλέκτη απορριμμάτων.

17.7.2 Γενικευμένη συνάρτηση υπολογισμού αθροίσματος

Να υλοποιηθεί συνάρτηση sum που λαμβάνει ως παράμετρο λίστα ενός οποιουδήποτε αριθμητικού τύπου και επιστρέφει το άθροισμα των στοιχείων της λίστας ως double. Προσθέστε main στην οποία ελέγξτε την sum με ArrayList<Integer> και LinkedList<Double>.

Λύση

Η τυπική παράμετρος της συνάρτησης θα πρέπει να είναι του τύπου List<? extends Number>. Με αυτήν την παράμετρο, η συνάρτηση μπορεί να δεχτεί οποιαδήποτε γενίκευση εφόσον υλοποιεί τη διεπαφή List. Επιπλέον, η λίστα μπορεί να εξειδικεύεται ως προς οποιονδήποτε τύπο κληρονομεί την αφηρημένη κλάση Number. Στην περίπτωση που σκεφτήκατε να ορίσετε την παράμετρο ως List<Number>, προσέξτε το εξής: Αυτή η παράμετρος επιτρέπει μόνο λίστες από Number. Βεβαίως μία λίστα από Number μπορεί να περιέχει ως στοιχεία αντικείμενα των παράγωγων κλάσεων. Ωστόσο, δεν είναι δυνατό να περάσουμε στη θέση της μία λίστα από Integer ή μία λίστα από Double.

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;

public class Calculator {

    public static double sum(List<? extends Number> list) {
        double sum = 0;
        Iterator<? extends Number> it = list.iterator();
        while (it.hasNext()) {
            sum += it.next().doubleValue();
        }
        return sum;
    }

    public static double product(List<? extends Number> list) {
        double product = 1;
        Iterator<? extends Number> it = list.iterator();
        while (it.hasNext()) {
            product *= it.next().doubleValue();
        }
        return product;
    }

    public static void main(String[] args) {
        List<Integer> li = new ArrayList<>();
        List<Double> ld = new LinkedList<>();
        List<Number> ln=new ArrayList<>();

        li.add(5);
        li.add(6);
```

```
            ld.add(5.2);
            ld.add(6.5);
            ln.add(5);
            ln.add(5.5);
            System.out.println(sum(li));
            System.out.println(sum(ld));
            System.out.println(sum(ln));
        }
    }
```

Κώδικας 17.14 *Η συνάρτηση sum(List<? extends Number>*

Από τη στιγμή που έχουμε προσδιορίσει σωστά τον τύπο της παραμέτρου, η υλοποίηση της συνάρτησης είναι απλή. Χρησιμοποιώντας τον Iterator που παίρνουμε από τη λίστα, διασχίζουμε όλα τα στοιχεία της και προσθέτουμε την πραγματική τιμή τους στον αθροιστή sum τον οποίο και επιστρέφουμε.

17.7.3 Κατασκευή πινάκων

Να υλοποιηθεί στατική συνάρτηση mkA με λίστα παραμέτρων μεταβλητού τύπου του παραμετροποιημένου τύπου Τ. Η συνάρτηση επιστρέφει έναν πίνακα τύπου Τ που περιέχει τα αντικείμενα που περνούν ως πραγματικές παράμετροι κατά την κλήση της.

Να προστεθεί ο ακόλουθος κώδικας στην main:

```
public static void main(String[] args) {
        String[] s = mkA("Jack", "John", "Mary");
        Integer[] k = mkA(1, 2, 3, 4, 5);
        for (int i = 0; i < s.length; i++) {
            System.out.println(s[i]);
            System.out.println(k[i]);
        }
    }
```

Λύση

Πρόκειται για πολύ απλή συνάρτηση. Το μόνο κρίσιμο σημείο είναι η σωστή ανάπτυξη της διεπαφής της συνάρτησης. Κατά τα λοιπά, η συνάρτηση απλώς επιστρέφει την παράμετρό της η οποία είναι πίνακας τύπου Τ και περιέχει όλες τις πραγματικές παραμέτρους.

```
public static <T> T[] mkA(T... a) {
    return a;
}
```

17.8 Ασκήσεις προς Λύση

17.8.1 Γενικευμένη δομή LIFO

Να υλοποιηθεί η γενικευμένη κλάση MyStack<E> που αναπαριστά μια δομή LIFO χωρίς να χρησιμοποιηθεί η προκαθορισμένη Stack<E>. Η κλάση πρέπει να υποστηρίζει τις λειτουργίες empty που επιστρέφει true αν αυτό το stack είναι άδειο, την push(E item) που εισάγει το item στην κορυφή της στοίβας και την pop που διαγράφει και επιστρέφει το item από την κορυφή της στοίβας.

17.8.2 Γενικευμένη δομή FIFO

Να υλοποιηθεί η γενικευμένη κλάση Queue<E> που αναπαριστά μια δομή FIFO. Η κλάση πρέπει να υποστηρίζει τις λειτουργίες empty που επιστρέφει true αν αυτή η ουρά είναι άδεια, την add(E item) που εισάγει το item στο τέλος της ουράς και την delete που διαγράφει και επιστρέφει το στοιχείο στην αρχή της ουράς.

17.8.3 Γενικευμένη συνάρτηση υπολογισμού γινομένου

Να υλοποιηθεί συνάρτηση product που λαμβάνει ως παράμετρο λίστα ενός οποιουδήποτε αριθμητικού τύπου και επιστρέφει το γινόμενο των στοιχείων της λίστας ως double. Προσθέστε main στην οποία ελέγξτε την product με ArrayList<Integer> και LinkedList<Double>.

Βιβλιογραφία

[1] “Generics, Inheritance, and Subtypes (The Java™ Tutorials > Learning the Java Language > Generics (Updated)).” https://docs.oracle.com/javase/tutorial/java/generics/inheritance.html (accessed Nov. 19, 2021).

Κεφάλαιο 18

Σύνοψη
Στην ενότητα αυτή παρουσιάζονται οι εκφράσεις λάμδα μαζί με τις λειτουργικές διεπαφές, οι αναφορές μεθόδων καθώς και η απομνημόνευση, μία τεχνική που περιορίζει των αριθμό των κλήσεων στις αναδρομικές συναρτήσεις και κάνει τον κώδικα πιο αποτελεσματικό.

Προαπαιτούμενη γνώση
Η ενσωμάτωση, η κληρονομικότητα και ο πολυμορφισμός στο αντικειμενοστρεφές μοντέλο με Java, οι αφηρημένες κλάσεις καθώς και οι βασικές προκαθορισμένες κλάσεις, δηλαδή οι String, StringBuilder, ArrayList, LinkedList, Stack, PriorityQueue, HashSet, HashMap, HashTree, LocalDate, Localtime, Period και Duration. Οι διεπαφές στην Java και οι γενικεύσεις.

Λέξεις κλειδιά
Λειτουργικές διεπαφές (functional interfaces), Εκφράσεις λάμδα (Lamda expressions), Αναφορές μεθόδων (method references), Απομνημόνευση (memoization).

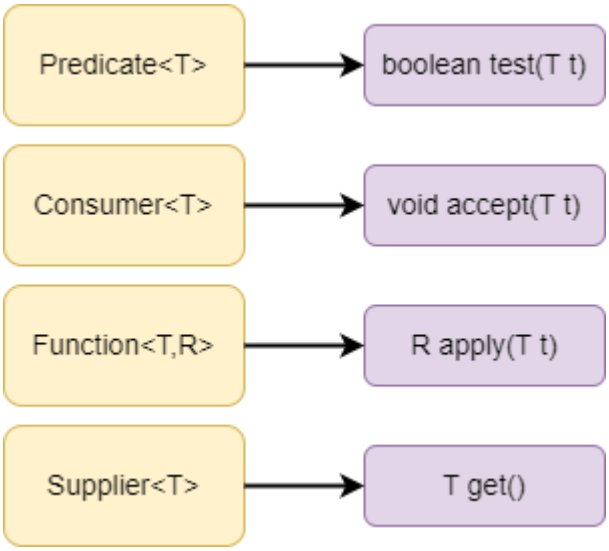
18 Ειδικά θέματα

18.1 Εκφράσεις Λάμδα

Συχνά χρειάζεται να περάσουμε ως παράμετρο σε μία συνάρτηση ένα αντικείμενο. Σε κάποιες περιπτώσεις είναι αντικείμενο μιας κλάσης που υλοποιεί μία διεπαφή. Σύμφωνα με όσα έχουμε ήδη μάθει, έχουμε δύο δυνατότητες. Να ορίσουμε την κλάση που υλοποιεί τη διεπαφή και στη συνέχεια το αντικείμενο ή να δημιουργήσουμε ένα αντικείμενο ανώνυμης κλάσης. Όμως αρκετά συχνά η διεπαφή έχει μία μόνο αφηρημένη μέθοδο. Επομένως, σε αυτές τις περιπτώσεις, το μόνο που είναι πράγματι αναγκαίο να κάνουμε είναι να ορίσουμε τον κώδικα της αφηρημένης μεθόδου. Θα ήταν μεγάλη ευκολία αν μπορούσαμε να περάσουμε ως παράμετρο κατευθείαν τον κώδικα που υλοποιεί την αφηρημένη μέθοδο και αποφεύγαμε τη δήλωση επώνυμης ή και ανώνυμης κλάσης. Αυτήν ακριβώς τη λειτουργικότητα μας παρέχουν οι εκφράσεις Λάμδα (Lamda expressions) που προστέθηκαν στην έκδοση 8 της Java.

18.1.1 Λειτουργικές διεπαφές

Λειτουργικές διεπαφές (functional interfaces) είναι αυτές που διαθέτουν μία μόνο αφηρημένη μέθοδο [1]. Βεβαίως, μια λειτουργική διεπαφή μπορεί να περιλαμβάνει πολλές στατικές ή προκαθορισμένες μεθόδους. Η Java διαθέτει ένα μεγάλο πλήθος προκαθορισμένων λειτουργικών διεπαφών. Ειδικά το πακέτο java.util.function αποτελείται αποκλειστικά από λειτουργικές διεπαφές, ενώ υπάρχουν πολλές τέτοιες διεπαφές διασκορπισμένες και στα υπόλοιπα πακέτα. Φυσικά, λειτουργικές διεπαφές μπορούν να οριστούν και από τον χρήστη.
Στο σχήμα 18.1 παρουσιάζονται τέσσερις βασικές προκαθορισμένες λειτουργικές διεπαφές που ανήκουν στο πακέτο java.util.function:



Σχήμα 18.1 *Τέσσερις βασικές λειτουργικές διεπαφές*

Η διεπαφή Predicate<T> δηλώνει την αφηρημένη μέθοδο boolean test(T t). Η Consumer<T> την αφηρημένη μέθοδο void accept(T t). Η Function<T,R> την R apply(T t) και η Supplier<T> την T get().
Στη συνέχεια θα δούμε πώς μπορεί να αξιοποιηθούν αυτές οι διεπαφές σε συνδυασμό με τις εκφράσεις lamda, ξεκινώντας από την Predicate<T>.
Ας υποθέσουμε πως χρειαζόμαστε να τυπώνουμε λίστες από αντικείμενα Person στις οποίες θα περιλαμβάνονται μόνο όσα πρόσωπα πληρούν κάποια ηλικιακά κριτήρια. Πιο συγκεκριμένα, έστω ότι θέλουμε να τυπώσουμε μία λίστα με όσα πρόσωπα είναι ηλικίας άνω των 50 ετών και μία λίστα με όσα πρόσωπα είναι από 40 έως 55 ετών. Μπορούμε να φτιάξουμε μία συνάρτηση που θα παίρνει ως παράμετρο μία λίστα με το σύνολο των προσώπων και ένα αντικείμενο Predicate<Person>. Εφόσον ένα πρόσωπο θα περνά επιτυχώς την test της Predicate θα τυπώνεται. Στον κώδικα 18.1 παρουσιάζεται μία τέτοια συνάρτηση:

```
static void prt(List<Person> list, Predicate<Person> choice) {
    Iterator<Person> it = list.iterator();
    while (it.hasNext()) {
        Person p = it.next();
        if (choice.test(p)) {
            System.out.println(p + " " + p.getAge());
        }
    }
}
```

Κώδικας 18.1 *Εκτύπωση λίστας προσώπων με κριτήρια*

Σύμφωνα με όσα γνωρίζουμε για να καλέσουμε την ptr ώστε να εκπληρώσουμε τον σκοπό μας χρειαζόμαστε δύο κλάσεις όπως δείχνει ο κώδικας 18.2.

```
class SelectorGreater50 implements Predicate<Person> {

    @Override
    public boolean test(Person t) {
        return t.getAge() > 50;
    }
}

class SelectorBetween40and55 implements Predicate<Person> {

    @Override
    public boolean test(Person t) {
       return t.getAge() >=40 && t.getAge()<=55;
    }
```

```
    }
```

Κώδικας 18.2 *Δύο κλάσεις που υλοποιούν τη διεπαφή Predicate*

Έχοντας στη διάθεσή μας τους κώδικες 18.1 και 18.2, μπορούμε να εκπληρώσουμε τον στόχο μας όπως δείχνει ο κώδικας 18.3.

```
public static void main(String[] args) {
    List<Person> list = loadData();
    prt(list, new SelectorGreater50());
    System.out.println("--------------------");
    prt(list, new SelectorBetween40and55());
}
```

Κώδικας 18.3 *Εκτύπωση προσώπων άνω των 50 ετών και εκτύπωση προσώπων από 40 έως και 55 ετών.*

Προσέξτε τώρα τα εξής: Καταρχάς, η διεπαφή Predicate είναι λειτουργική, δηλαδή διαθέτει μία μόνο αφηρημένη μέθοδο, την test. Επομένως, για να ορίσουμε μία κλάση που υλοποιεί τη διεπαφή εκείνο που είναι αναγκαίο είναι ο ορισμός της test. Στη μία περίπτωση ο ορισμός της test δίνεται από τον κώδικα

```
return t.getAge() > 50;
```

και στην άλλη περίπτωση από τον κώδικα

```
return t.getAge() >=40 && t.getAge()<=55;
```

Γιατί όμως να είμαστε αναγκασμένοι να ορίσουμε τις κλάσεις SelectorGreater50 και SelectorBetween40and55, αφού η μόνη πληροφορία που προσθέτουμε σε κάθε κλάση περιορίζεται στον αντίστοιχο κώδικα της test; Εδώ είναι που μας βοηθούν οι εκφράσεις λάμδα κάνοντας τον κώδικά μας συνοπτικό και κομψό, ενώ αυξάνουν την παραγωγικότητά μας. Οι εκφράσεις λάμδα λοιπόν μας επιτρέπουν να περάσουμε στη θέση ενός αντικειμένου λειτουργικής διεπαφής μόνο τον κώδικα που υλοποιεί τη μοναδική αφηρημένη μέθοδο της διεπαφής.
Έτσι, ο κώδικας 18.3 μπορεί να μεταγραφεί με τη χρήση των εκφράσεων λάμδα όπως δείχνει ο κώδικας 18.4.

```
public static void main(String[] args) {
    List<Person> list = loadData();
    prt(list, t->t.getAge()>50);
    System.out.println("----------");
    prt(list, t->t.getAge()>=40 && t.getAge()<=55);
}
```

Κώδικας 18.4 *Χρήση των εκφράσεων λάμδα*

Στην πρώτη κλήση της prt στον κώδικα 18.4, δίνουμε τον κώδικα που υλοποιεί την test για ηλικίες άνω των 50 ετών. Η test γνωρίζουμε πως απαιτεί ως παράμετρο ένα Person, γεγονός που είναι γνωστό και στον μεταγλωττιστή, επομένως δεν είναι αναγκαίο να ορίσουμε τον τύπο της παραμέτρου. Επειδή όμως είναι αναγκαίο να αναφερθούμε σε αυτήν, θα πρέπει να της δώσουμε ένα όνομα. Το όνομα αυτό προηγείται του συμβόλου ->, ενώ ο κώδικας που ακολουθεί το σύμβολο -> είναι ο κώδικας που υλοποιεί την αφηρημένη μέθοδο.
Με τη βοήθεια των εκφράσεων λαμδα είναι εύκολο να τυπώσουμε λίστες προσώπων με οποιαδήποτε κριτήρια ηλικιακά ή μη. Για παράδειγμα, μπορούμε να χρησιμοποιήσουμε την prt για να τυπώσουμε όλα τα πρόσωπα της αρχικής λίστας όπως δείχνει ο κώδικας που ακολουθεί:

```
prt(list, p->true);
```

ή να τυπώσουμε όλα τα πρόσωπα που έχουν επώνυμο Smith

```
prt(list, p->p.getsName().equals("Smith"));
```

Στη συνέχεια δίνουμε παραδείγματα αξιοποίησης των υπόλοιπων τριών λειτουργικών διεπαφών που αναφέρονται σε αυτήν την ενότητα.
Όπως θα έγινε αντιληπτό, η διεπαφή Predicate έχει ως στόχο να λαμβάνει μία παράμετρο, να την αξιολογεί και να επιστρέφει true ή false ανάλογα με κάποια κριτήρια. Αντίθετα, η διεπαφή Consumer έχει ως στόχο να επεξεργάζεται την παράμετρο της αφηρημένης μεθόδου της. Ο κώδικας 18.5 δίνει ένα σχετικό παράδειγμα:

```
import java.util.Iterator;
import java.util.List;
import java.util.function.Consumer;

public class Consumers {

    static void process(List<Person> list, Consumer<Person> consumer) {
        Iterator<Person> it = list.iterator();
        while (it.hasNext()) {
            Person p = it.next();
            consumer.accept(p);
        }
    }

    public static void main(String[] args) {
        List<Person> list = loadData();
        prt(list, p -> true);
        System.out.println("-----------------------");
        process(list, p -> p.setBirthday(p.getBirthday().minusYears(20)));
        prt(list, p -> true);

    }
}
```

Κώδικας 18.5 *Παράδειγμα χρήσης της Consumer*

Η συνάρτηση process του κωδικα 18.5 διασχίζει μια λίστα από αντικείμενα Person και σε κάθε αντικείμενο εφαρμόζει την επεξεργασία της Consumer. Η κλήση της στην main έχει ως αποτέλεσμα τη μεταβολή της ημερομηνίας γέννησης για όλα τα πρόσωπα της λίστας. Πιο συγκεκριμένα, μετά την κλήση της process στην main, όλα τα πρόσωπα της λίστας εμφανίζονται γηραιότερα κατά 20 έτη.
Ακολουθεί παράδειγμα χρήσης της διεπαφής Function:

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.function.Function;
import static tmp.FunctionalInterfaces.Predicates.prt;

public class Functions {

    static List<Person> getOlders(List<Person> list,
Function<Person,Boolean> f) {
        List<Person> rVal=new ArrayList<>();
        Iterator<Person> it=list.iterator();
        while (it.hasNext()) {
            Person p=it.next();

            if (f.apply(p)) {
               rVal.add(p);
            }
        }
        return rVal;
    }
```

```
    public static void main(String[] args) {
        List<Person> list = loadData();
        Function<Person, Boolean> ageLimit=p->p.getAge()>50;
        prt(list,p->true);
        System.out.println("-----------------");
        List<Person> olders=getOlders(list, ageLimit);
        prt(olders,p->true);
    }
}
```

Κώδικας 18.6 *Παράδειγμα χρήσης της Function*

Η Function είναι παραμετροποιημένη ως προς δύο τύπους. Η αφηρημένη μέθοδός της, apply, λαμβάνει τον ένα τύπο ως παράμετρο και επιστρέφει τον άλλο. Στον κώδικα 18.6, χρησιμοποιείται Function<Person, Boolean>. Στην main με τον ορισμό της ageLimit, επιδεικνύεται η δυνατότητα ορισμού εκφράσεων λάμδα ως απλές μεταβλητές. Η ageLimit περνά στην getOlders. Η getOlders προσθέτει κάθε Person για το οποίο η apply επιστρέφει true στην λίστα rVal. Τελικά η getOlders επιστρέφει μια λίστα με όλα τα πρόσωπα άνω των 50 ετών. Στη συνέχεια, στον κώδικα 18.7, με τη βοήθεια της διεπαφής Supplier δημιουργούμε μία μεταβλητή ικανή να παράγει τυχαίους ακέραιους:

```
import java.util.Random;
import java.util.function.Supplier;

public class Suppliers {

    public static void main(String[] args) {
        Supplier<Integer> gen = () -> new Random().nextInt();
        for (int i = 0; i < 10; i++) {
            System.out.println(gen.get());
        }
    }
}
```

Κώδικας 18.7 *Παράδειγμα χρήσης της Supplier*

Προσέξτε πως καθώς η αφηρημένη μέθοδος get δεν δέχεται παραμέτρους, στον ορισμό της έκφρασης λάμδα έχουμε δώσει κενή λίστα παραμέτρων.

18.1.2 Το συντακτικό των εκφράσεων λάμδα

Μία έκφραση λάμδα συντάσσεται ως ακολούθως:

```
λίστα παραμέτρων βέλος κώδικας
```

Η λίστα παραμέτρων μπορεί να είναι κενή ή να περιλαμβάνει μία ή περισσότερες παραμέτρους. Στην περίπτωση που περιλαμβάνει μία μόνο παράμετρο, οι παρενθέσεις μπορούν να παραλειφθούν.

Ο κώδικας μπορεί να είναι μία απλή έκφραση ή ένα μπλοκ κώδικα. Στην περίπτωση που είναι μπλοκ κώδικα και εφόσον η αφηρημένη μέθοδος δεν είναι void, η επιστροφή θα πρέπει να γίνει με χρήση της δεσμευμένης λέξης return.

Οι εκφράσεις λάμδα είναι στην ουσία ένα είδος ανώνυμων συναρτήσεων [2]. Προσέξτε το επόμενο παράδειγμα όπου ταξινομούμε μία λίστα από πρόσωπα πρώτα ως προς το id τους και μετά ως προς επώνυμο και όνομα.

```
import java.util.List;

public class ComparatorLamda {

    public static void main(String[] args) {
        List<Person> list = PolymorphismDemo.loadData();
        list.sort((p1, p2) -> {
```

```
            int id1 = Integer.decode(p1.getId());
            int id2 = Integer.decode(p2.getId());

            return id1 - id2;
        });
        prt(list, p -> true);
        System.out.println("--------------------");
        list.sort((p1, p2) -> {
            String sName1 = p1.getsName();
            String sName2 = p2.getsName();
            String fName1 = p1.getfName();
            String fName2 = p2.getfName();
            int rVal = sName1.compareTo(sName2);
            if (rVal == 0) {
                return fName1.compareTo(fName2);
            }
            return rVal;
        });
        prt(list, p -> true);
    }
```

Κώδικας 18.8 *Ταξινόμηση με εκφράσεις λάμδα*

18.1.3 Αναφορές μεθόδων

Εφόσον έχουμε στη διάθεσή μας μία μέθοδο που είναι ταυτόσημη ως προς τη λειτουργικότητα με τον κώδικα μίας έκφρασης λάμδα, τότε η έκφραση λάμδα μπορεί να αντικατασταθεί από την αναφορά της μεθόδου (method reference) [3].

```
import java.util.List;

public class MethodReferences {

    static boolean isOlder(Person p) {
        return p.getAge() > 50;
    }

    static boolean between40And55(Person p) {
        return p.getAge() >= 40 && p.getAge() <= 55;
    }

    public static void main(String[] args) {
        List<Person> list = loadData();
        prt(list, MethodReferences::isOlder);
        System.out.println("----------");
        prt(list, MethodReferences::between40And55);
    }
}
```

Κώδικας 18.9 *Χρήση αναφορών μεθόδων*

Ο κώδικας 18.9 περιλαμβάνει δύο boolean μεθόδους, την isOlder και την between40And55. Με δεδομένες αυτές τις μεθόδους, ο κώδικας 18.4 έχει μετασχηματιστεί στην main του κώδικα 18.9. Με άλλα λόγια, η έκφραση λάμδα

```
t->t.getAge()>50
```

αντικαταστάθηκε από την αναφορά

```
MethodReferences::isOlder
```

και η έκφραση

```
t->t.getAge()>=40 && t.getAge()<=55
```

αντικαταστάθηκε από την αναφορά

```
MethodReferences::between40And55
```

Στο παράδειγμα αυτό οι εκφράσεις λάμδα απαιτούν ως παράμετρο ένα αντικείμενο τύπου Person, την ίδια παράμετρο απαιτούν και οι αναφορές μεθόδων που αντικατέστησαν τις εκφράσεις λάμδα. Επίσης τόσο οι εκφράσεις λάμδα και οι αναφορές μεθόδων επιστρέφουν boolean τιμή. Είναι αυτονόητο πως για να επιτρέψει ο μεταγλωττιστής αντικατάσταση μιας έκφρασης λάμδα, θα πρέπει τόσο οι παράμετροί της όσο και η τιμή επιστροφής της να είναι συμβατές με αυτές της μεθόδου αναφοράς.
Στο παράδειγμα αυτό χρησιμοποιήθηκαν στατικές μέθοδοι. Μπορεί βέβαια να χρησιμοποιηθούν και μέθοδοι στιγμιότυπου. Για παράδειγμα, η πρώτη γραμμή της main στον κώδικα 18.7, δηλαδή ο κώδικας

```
Supplier<Integer> gen = () -> new Random().nextInt();
```

είναι ισοδύναμος με τον κώδικα

```
Supplier<Integer> gen = new Random()::nextInt;
```

Ειδικά, οι αναφορές σε δημιουργούς γίνονται με τη χρήση της δεσμευμένης λέξης new. Για παράδειγμα, ο κώδικας

```
Supplier<String> newString = () -> new String();
```

μπορεί να αντικατασταθεί από τον κώδικα

```
Supplier<String> newString = String::new;
```

Όταν όμως ο δημιουργός διαθέτει παραμέτρους, τότε δεν μπορεί να χρησιμοποιηθεί η διεπαφή Supplier, καθώς η μέθοδος get είναι χωρίς παραμέτρους. Μπορεί όμως να χρησιμοποιηθεί η διεπαφή Function, όπως επιδεικνύει ο κώδικας που ακολουθεί:

```
Function<String, String> s=String::new;
System.out.print(s.apply("Hello"));
System.out.println(s.apply(" Mike"));
```

18.2 Απομνημόνευση

Η απομνημόνευση (memoization) είναι μία τεχνική που εφαρμόζεται σε συνδυασμό με την αναδρομή με σκοπό να περιορίσει το πλήθος των αναδρομικών κλήσεων όπου αυτό είναι δυνατό.
Ας πάρουμε για παράδειγμα τον υπολογισμό του ν-οστού όρου της ακολουθίας Fibonacci. Ο κώδικας 9.4 υπολογίζει αναδρομικά τον ν-οστό όρο. Επίσης, το σχήμα 9.3 μας δίνει μία εικόνα των αναδρομικών κλήσεων. Όπως φαίνεται στο σχήμα 9.3, κατά τον υπολογισμό του όρου με ν=4, η κλήση για n=2 επαναλαμβάνεται δύο φορές. Αν επιχειρήσουμε να υπολογίσουμε τον 20ό όρο της ακολουθίας και μετρήσουμε τις αναδρομικές κλήσεις (Ενότητα 9.7, Άσκηση 5) θα διαπιστώσουμε πως απαιτούνται 21891 αναδρομικές κλήσεις. Αν όμως βρούμε έναν τρόπο ώστε οι τιμές της ακολουθίας να υπολογίζονται μόνο μία φορά η κάθε μία, οι αναδρομικές κλήσεις μειώνονται δραματικά.

```
import java.util.HashMap;

public class Fibonacci {
```